CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler 0x önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Kerem Yılmaz
Öğrenci numarası (SID)	5001240012
E-posta	kerem.yilmaz.1@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G2
Seçilen ISA ve sürümü (ör. RV32IM)	RV32IM
Derleyici ve sürümü	RISC-V GCC 16.1.0
Simülatör / araçlar	RARS 1.6 (RISC-V Assembler and Runtime Simulator), RARS Instruction Counter
Git deposu adresi	github.com/Kerem3826/CMPE3209-RV32IM-G2

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001240012%10	2
D2	SID'in son iki hanesi	5001240012%100	12
S	SID'in hanelerinin toplamı	5+0+0+1+2+4+0 +0+1+2	15
N	24 + (D2 mod 25)	24 + (12%25)	36

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı		256 / 16 / Direct / 16 / -
Write policy	d çift → write-back, d tek → write-through		write-back
k	(S mod 7) + 3	(15%7) + 3	4
Interrupt türü	S çift → Timer, S tek → I/O		I/O
f	1.0 + 0.1 × d GHz (varsayım)	1 + 0.1 * 2	1.2 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	20 + 5 * (2%4)	1 / 30 (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (15%7)	3 (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	P = 50 + (5001240012 mod 10)	52

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	4	$\log_2(16)$
Line (r) veya set (d)	4	$\log_2(16)$
Tag (s − r veya s − d)	24	32-(4+4)
Toplam	32	4 + 4+24

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☒ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> *Verilen değerlere göre N=36 boyutunda global bir arr not dizisi oluşturulur. Bu değerler oluşturulduktan sonra hangi aralığa düştüklerine göre bucket değerleri arttırılmaya başlanır. Değerler ortalamayı bulmak için toplanır. Geçen kişi sayısını bulmak için dizideki değerleri verilen geçer not değeriyle karşılaştıran passCheck fonksiyonu tanımlanır ve dizi üzerinde kullanılarak pass_count değeri arttırılır. Ortalama hesaplanır*

> *ve ortalamanın altında kalan notlar dizinin tekrar üzerinden geçerek 8. bitleri işaretlenir ve below_count değeri arttırılır.*

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

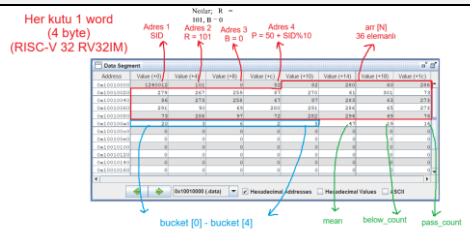
```
1: /* Kerem Yılmaz, 5001240012, G2 */
 2: #define N 36
 3: const int SID = 1240012;
 4: const unsigned int R = 101;
 5: const unsigned int B = 0;
 6: const unsigned int P = 50 + (SID%10);
 7:
 8:
 9: int arr[N];
10:
11: unsigned int bucket[5];
12: unsigned int mean;
13: unsigned int below_count = 0;
14: unsigned int pass_count = 0;
15:
16: int passCheck(unsigned int r){
17:     if(r>=P){
18:         return 1;
19:     }
20:     else return 0;
21: }
22:
23: int main(){
24:
25:     unsigned int x = SID;
26:     unsigned int sum = 0;
27:     unsigned int i;
28:
29:     for (i = 0; i < N; i++){
30:         x = 1664525 * x + 1013904223;
31:         arr[i] = ((x >> 16) % R) + B;
32:
33:         if(arr[i]<50) bucket[0]++;
34:         else if(arr[i]<65) bucket[1]++;
35:         else if(arr[i]<75) bucket[2]++;
36:         else if(arr[i]<85) bucket[3]++;
37:         else bucket[4]++;
38:
39:         sum += arr[i];
40:     }
41:
42:     for (i = 0; i < N; i++){
43:         pass_count += passCheck(arr[i]);
44:     }
45:
46:     mean = sum/N;
47:
48:     for (i = 0; i < N; i++){
49:
50:         if(arr[i] < mean){
51:             arr[i] = arr[i] | 0x100;
```

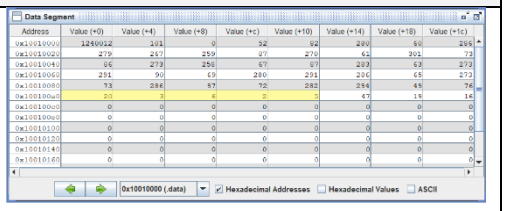
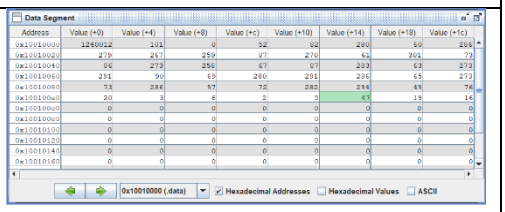
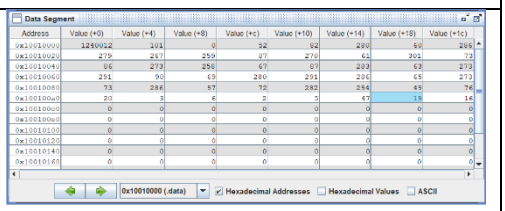
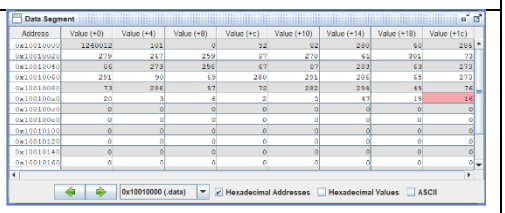
```
52:             below_count++;
53:         }
54:     }
55:
56:     return 0;
57: }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	9, 29, 42, 48	Global arr[36] dizisi üzerinde 1. geçişte elemanlar üretilip kategorize edilir, 2. geçişte passCheck fonksiyonu çağırılır ve geçenlerin sayısı belirlenir. 3. geçişte ise ortalama altındakiler işaretlenir.
Y2	Koşullu dallanma	17, 33, 50	Notların aralıklara ayrılması, baraj kontrolü (r >= P) ve ortalama altı tespiti için if-else yapıları kullanılmıştır.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	43	passCheck(arr[i]) biçiminde parametre alan ve int (1/0) döndüren fonksiyon çağrılmıştır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	30, 31, 51	LCG üretecindeki çarpma/kaydırma işlemleri ile işaretleme adımındaki arr[i] = arr[i] | 0x100 bitsel VEYA işlemi yapılmıştır.
Y5	Dizi elemanına store	31, 51	İşaretlenen yeni not değerleri (arr[i] = ...) ifadeleriyle belleğe yazılmıştır (sw).
Y6	const int SID ve hesapta kullanımı	3, 6, 25	const unsigned int SID tanımlanmış ve P = 50 + (SID % 10) geçme barajı hesabında kullanılmıştır.
Y7	Sonucun tutulduğu global değişken(ler)	11-14, 43, 46, 52	bucket[5], mean, below_count ve pass_count değişkenleri global olarak tanımlanıp hesaplanmıştır.

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
			

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
bucket[0..4]	0x100100A0 – 0x100100B0	20, 3, 6, 2, 5	
mean	0x100100B4	47	
below_count	0x100100B8	19	
pass_count	0X100100BC	16	

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	16-21
Başlangıç ve bitiş adresi (hex)	0x00400328 – 0X00400364
Dinamik instruction sayısı	15
Hangi döngü iterasyonu (ör. i = 3)	arr[0] = 82 için çağırılan passCheck

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (`-O0`; listenin tamamı depoda `asm_O0.s`):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
3	const int *SID* = 1240012;	.data SID: .word 1240012	0x10010000
4	const unsigned int *R* = 101;	R: .word 101	0x10010004
5	const unsigned int *B* = 0;	B: .space 4	0x10010008
6	const unsigned int *P* = 50 + (*SID*%10);	P: .word 52	0x1001000C
9	int arr[N];	arr: .space 144	0x10010010 – 0x1001009C
11	unsigned int bucket[5];	bucket: .space 20	0x100100A0 – 0x100100B0
12	unsigned int mean;	mean: .space 4	0x100100B4
13	unsigned int below_count = 0;	below_count: .space 4	0x100100B8
14	unsigned int pass_count = 0;	pass_count: .space 4	0x100100BC
16	int passCheck(unsigned int r){	passCheck: addi sp,sp,-32 sw ra,28(sp) sw s0,24(sp) addi s0,sp,32 sw a0,-20(s0)	0x00400328 – 0x00400338
17	if(r>=P){	li a5,52 lw a4,-20(s0)	0x0040033C – 0x00400344

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		bltu a4,a5,.L2	
18	return 1;	li a5,1 j .L3	0x00400348 – 0x0040034C
20	else return 0;	.L2: li a5,0	0x00400350
21	}	.L3: mv a0,a5 lw ra,28(sp) lw s0,24(sp) addi sp,sp,32 jr ra	0x00400354 – 0x00400364
23	int main(){	.text .globl main main: addi sp,sp,-32 sw ra,28(sp) sw s0,24(sp) addi s0,sp,32	0x00400000 – 0x0040000C
25	unsigned int x = *SID*;	li a5,1241088 addi a5,a5,-1076 sw a5,-20(s0)	0x00400010 – 0x0040001C
26	unsigned int sum = 0;	sw zero,-24(s0)	0x00400020
29	for (i = 0; i < N; i++){	sw zero,-28(s0) j .L5	0x00400024 – 0x00400028
30	x = 1664525 * x + 1013904223;	.L11: lw a4,-20(s0) li a5,1662976	0x0040002C – 0x00400050

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		addi a5,a5,1549 mul a4,a4,a5 li a5,1013903360 addi a5,a5,863 add a5,a4,a5 sw a5,-20(s0)	
31	arr[i] = ((x >> 16) % *R*) + *B*;	lw a5,-20(s0) srli a4,a5,16 li a5,101 remu a5,a4,a5 li a4,0 add a5,a5,a4 mv a3,a5 la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 sw a3,0(a5)	0x00400054 – 0x00400084
33	if(arr[i]<50) bucket[0]++;	la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a4,0(a5) li a5,49 bgt a4,a5,.L6 la a5,bucket lw a5,0(a5)	0x00400088 – 0x004000C4

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		addi a4,a5,1 la a5,bucket sw a4,0(a5) j .L7	
34	else if(arr[i]<65) bucket[1]++;	.L6: la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a4,0(a5) li a5,64 bgt a4,a5,.L8 la a5,bucket lw a5,4(a5) addi a4,a5,1 la a5,bucket sw a4,4(a5) j .L7	0x004000C8 – 0x00400104
35	else if(arr[i]<75) bucket[2]++;	.L8: la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a4,0(a5) li a5,74 bgt a4,a5,.L9 la a5,bucket	0x00400108 – 0x00400144

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		lw a5,8(a5) addi a4,a5,1 la a5,bucket sw a4,8(a5) j .L7	
36	else if(arr[i]<85) bucket[3]++;	.L9: la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a4,0(a5) li a5,84 bgt a4,a5,.L10 la a5,bucket lw a5,12(a5) addi a4,a5,1 la a5,bucket sw a4,12(a5) j .L7	0x00400148 – 0x00400184
37	else bucket[4]++;	.L10: la a5,bucket lw a5,16(a5) addi a4,a5,1 la a5,bucket sw a4,16(a5)	0x00400188 – 0x004001A0
39	sum += arr[i];	.L7: la a4,arr	0x004001A4 – 0x004001C8

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a5,0(a5) mv a4,a5 lw a5,-24(s0) add a5,a5,a4 sw a5,-24(s0)	
40	}	lw a5,-28(s0) addi a5,a5,1 sw a5,-28(s0)	0x004001CC – 0x004001E0
42	for (i = 0; i < N; i++){	sw zero,-28(s0) j .L12	0x004001E4 – 0x004001E8
43	pass_count += passCheck(arr[i]);	.L13: la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a5,0(a5) mv a0,a5 call passCheck mv a5,a0 mv a4,a5 la a5,pass_count lw a5,0(a5) add a4,a4,a5 la a5,pass_count	0x004001EC – 0x00400230

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		sw a4,0(a5)	
44	}	lw a5,-28(s0) addi a5,a5,1 sw a5,-28(s0) .L12: lw a4,-28(s0) li a5,35 bleu a4,a5,.L13	0x00400234 – 0x00400248
46	mean = sum/N;	lw a4,-24(s0) li a5,954437632 addi a5,a5,-455 mulhu a5,a4,a5 srli a4,a5,3 la a5,mean sw a4,0(a5)	0x0040024C – 0x0040026C
48	for (i = 0; i < N; i++){	sw zero,-28(s0) j .L14	0x00400270 – 0x00400274
50	if(arr[i] < mean){	.L16: la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a5,0(a5) mv a4,a5 la a5,mean lw a5,0(a5) bgeu a4,a5,.L15	0x00400278 – 0x004002A0

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
51	arr[i] = arr[i] | 0x100;	la a4,arr lw a5,-28(s0) slli a5,a5,2 add a5,a4,a5 lw a5,0(a5) ori a4,a5,256 la a3,arr lw a5,-28(s0) slli a5,a5,2 add a5,a3,a5 sw a4,0(a5)	0x004002A4 – 0x004002D4
52	below_count++;	la a5,below_count lw a5,0(a5) addi a4,a5,1 la a5,below_count sw a4,0(a5)	0x004002D8 – 0x004002F0
54	}	.L15: lw a5,-28(s0) addi a5,a5,1 sw a5,-28(s0) .L14: lw a4,-28(s0) li a5,35 bleu a4,a5,.L16	0x004002F4 – 0x00400308
56	return 0;	li a5,0 mv a0,a5	0x0040030C – 0x00400310
57	}	lw ra,28(sp)	0x00400314 – 0x00400324

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		lw s0,24(sp) addi sp,sp,32 li a7,10 ecall	

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00400000	872 byte	Yürütülebilir makine komutları (main ve passCheck fonksiyonlarının assembly çıktıları)
.data / .bss	0x10010000	192 byte	Global sabitler ve değişkenler (SID, R, B, P, arr, bucket, mean, below_count, pass_count)

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

> *Von Neumann mimarisinde program komutları (.text'te bulunan) ve veriler (.data'da bulunan) ayrı fiziksel bellekler yerine tek bir birleşik bellek alanında (RAM'de) saklanır. Burada .text (0x00400000) ve .data (0x10010000) farklı adres aralıklarına bölünmüş olsa da işlemci aynı adres yolu (bus) üzerinden hem koda hem de veriye erişmektedir.*

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	15 (Dinamik) 16 (Statik)	3 (Dinamik ve Statik)	-O0 derlemesinde gereksiz stack çerçevesi (prologue/epilogue), belleğe yazıp-okuma adımları ve dallanma (bltu, j) üretilir. -O1'de derleyici mantıksal karşılaştırmayı dallanmasız (branchless) sltiu ve seqz komutlarına indirger, stack kullanımını tamamen kaldırır.
Load sayısı	3 (lw)	0	-O0'da a0 parametresi stack'ten okunur (lw a4) ve fonksiyondan çıkarken ra, s0 geri yüklenir. -O1'de veri

Ölçü	-O0	-O1	Farkın nedeni
			doğrudan a0 yazmacı üzerinde işlendiği için bellekten okuma (load) yapılmaz.
Store sayısı	3 (sw)	0	-O0'da fonksiyon girişinde ra, s0 ve a0 stack belleğe kaydedilir. -O1'de stack çerçevesi (stack frame) oluşturulmadığı için belleğe yazma (store) yapılmaz.
Kullanılan register sayısı	6 (sp, ra, s0, a0, a4, a5)	2 (a0, ra)	-O0'da ara değerler ve adresleme için çok sayıda yazmaç (s0, a4, a5, sp, ra, a0) harcanır. -O1'de derleyici yazmaç tahsisi (register allocation) optimizasyonu ile işlemi sadece girdi/çıktı yazmacı a0 üzerinde bitirir (ret komutu ise dönüş için ra kullanır).

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1	0x00400010	li a5, 1241088 (lui x15, 303)	U	imm[31:12] = 00000000000001001111, rd = 01111, opcode = 0110111	0x0012F7B7	B7 F7 12 00	Evet
2	0x00400018	addi a5, a5, -1076	I	imm[11:0]= 10111100110 0, rs1=01111, funct3=000,	0xBCC78793	93 87 C7 BC	Evet

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
				rd=01111, opcode=0010011			
3	0x0040032C	sw ra, 28(sp)	S	imm[11:5]=0000000, rs2=00001, rs1=00010, funct3=010, imm[4:0]=11100, opcode=0100011	0x00112E23	23 2E 11 00	Evet
4	0x00400340	lw a4, -20(s0)	I	imm[11:0]=111111101100, rs1=01000, funct3=010, rd=01110, opcode=0000011	0xFEC42703	03 27 C4 FE	Evet
5	0x00400344	bltu a4, a5, .L2	B	imm[12]=0, imm[10:5]=000000, rs2=01111, rs1=01110, funct3=110, imm[4:1]=0110, imm[11]=0, opcode=1100011	0x00F76663	63 66 F7 00	Evet
6	0x0040034C	j .L3 (jal x0, 8)	J	imm[20]=0, imm[10:1]=0000000100, imm[11]=0, imm[19:12]=00000000, rd=00000, opcode=1101111	0x0080006F	6F 00 80 00	Evet

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
7	0x00400354	mv a0, a5 (add x10, x0, x15)	R	funct7=0000000, rs2=01111, rs1=00000, funct3=000, rd=01010, opcode=0110011	0x00F00533	33 05 F0 00	Evet
8	0x00400208	call passCheck (auipc x6, 0)	U	imm[31:12]=00000000000000000000, rd=00110, opcode=0010111	0x00000317	17 03 00 00	Evet
9	0x0040020C	call passCheck (jalr x1, x6, 288)	I	imm[11:0]=000100100000, rs1=00110, funct3=000, rd=00001, opcode=1100111	0x120300E7	E7 00 03 12	Evet
10	0x00400364	jr ra (jalr x0, x1, 0)	I	imm[11:0]=000000000000, rs1=00001, funct3=000, rd=00000, opcode=1100111	0x00008067	67 80 00 00	Evet

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): #1, #2
- ☒ Koşullu branch: #5
- ☒ Load: #4
- ☒ Store: #3
- ☒ Fonksiyon çağrısı: #8, #9

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

> *Öğrenci numarası SID = 1240012 (0x0012E9EC) 32 bitlik bir sabittir. RISC-V I-Type komutlarındaki (addi) anlık değer (immediate) alanı 12 bit işaretli ([-2048, +2047]) olduğu için bu sabit tek bir komuta sığmaz.Değer iki komuta bölünür: Alt 12 bit 0x9EC desimal -1076'ya karşılık gelir. addi komutu bu 12 biti işaretiyle genişlettiği (sign-extension) için eklenen negatif değeri telafi etmek üzere üst 20 bit 1 artırılarak düzeltilir: 0x0012E + 1 = 0x0012F (303).Böylece derleyici önce lui a5, 303 (303 * 4096 = 1241088) komutuyla üst 20 biti yükler, ardından addi a5, a5, -1076 (1241088 - 1076 = 1240012) işlemiyle sabiti tam olarak elde eder.*

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

> *Branch Komutunun Adresi: 0x00400344 (bltu a4, a5, .L2)*
> *Hedef Adres (.L2): 0x00400350 (li a5, 0)*
> *Offset (Bayt Cinsinden): Hedef Adres - Mevcut Adres = 0x00400350 - 0x00400344 = +12 byte.*
>
> *RISC-V B-type komutlarında komut boyutu her zaman 2'nin katı olduğundan (en düşük bit 0 varsayılır), 12 baytlık offset 2'ye bölünerek 13-bitlik binary değere dönüştürülür:*
>
> *$12 / 2 = 6 = 000000000110_2$.*
>
> *Bitler B-type formatına dağıtılır: imm[12]=0, imm[10:5]=000000, imm[4:1]=0110, imm[11]=0. Bu değerler opkod (1100011), funct3 (110) ve x14, x15 yazmaçlarıyla birleşerek 0x00F76663 makine kodunu oluşturur.*

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

>

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										
16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 ADD B,A ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			
MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	

Soru	Cevap
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`.

Ölçü	Değer
Kullanılan derleme (`-O0` / `-O1`)	
Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			
Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		
T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	$Ic \times CPI \times \tau$		
MIPS rate	$f / (CPI \times 10^6)$		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	$Ic \times [p + (m \times k)] \times \tau$		
T, cache var (k = T_s)	$Ic \times [p + (m \times k)] \times \tau$		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	
Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = $1 / [(1 - f) + f / Nc]$	
$Nc \to \infty$ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Gemini 3.6 Flash	1	Assembly kodunu RARS'da çalışır hale getirmek	… bu assembly kodunu RARS'da çalışacak hale gelecek şekilde düzenle	RARS üzerinden kodu asemble edip çalıştırarak.
Gemini 3.6 Flash	1	GitHub reposu için README.md metni oluşturma	… Proje dosyalarım ve projenin amacı bunlar, GitHub'a yükleyeceğim README.md dosyam için bir metin oluştur	Okuyup düzeltilmesi gereken noktaları elle düzelterek.
Gemini 3.6 Flash	2	Soru açıklamaları için yazılan metinlerin formal hale getirilmesi	… Bu açıklamayı, hocamın anlayacağı şekilde resmileştir.	Yazdığı metinleri gözden geçirip yanlış bir bilgi var mı diye kontrol ederek.
Gemini 3.6 Flash	2	Objdump ile uyumlu mu kontrolü	… Bu dosyalara bakarak yazdığım hex değerleri ile objdumpta çıkan hex değerleri	Objdump dosyasını açıp değerlerden birkaçını kendim kontrol ederek

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
			uyumlu mu değil mi diye kontrol et.	

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Kerem Yılmaz	5001240012	08/11/2026	

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza